ENTERPRISE DIGITAL BANKING SYSTEM

03

Flow and Interaction Diagrams

Data Flow Diagrams (levels 0-2), sequence diagrams, activity diagrams and state diagrams - what happens when a user performs an action.

Document	03 — Flow and Interaction Diagrams
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Data Flow Diagrams
2. 2. Sequence Diagrams — Authentication
3. 3. Sequence Diagrams — Money Movement
4. 4. Sequence Diagrams — Beneficiary, Loan and EMI
5. 5. Activity Diagrams
6. 6. State Diagrams

1. Data Flow Diagrams

DFD notation used: rectangles = external entities, rounded boxes = processes, cylinders = data stores, arrows = data flows (labelled with the data that moves).

1.1 Level 0 — Context diagram

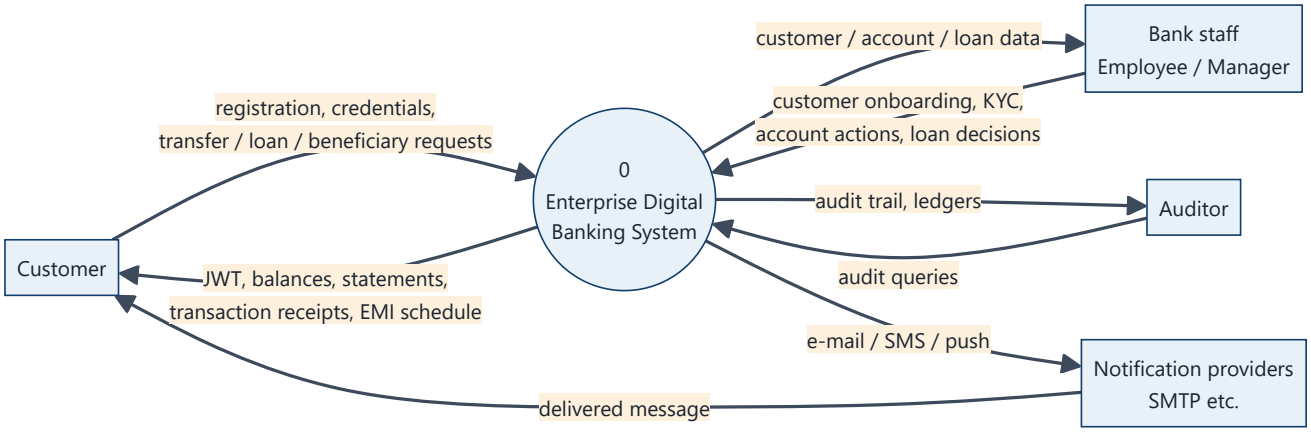


Figure 1 — DFD level 0

1.2 Level 1 — Main processes

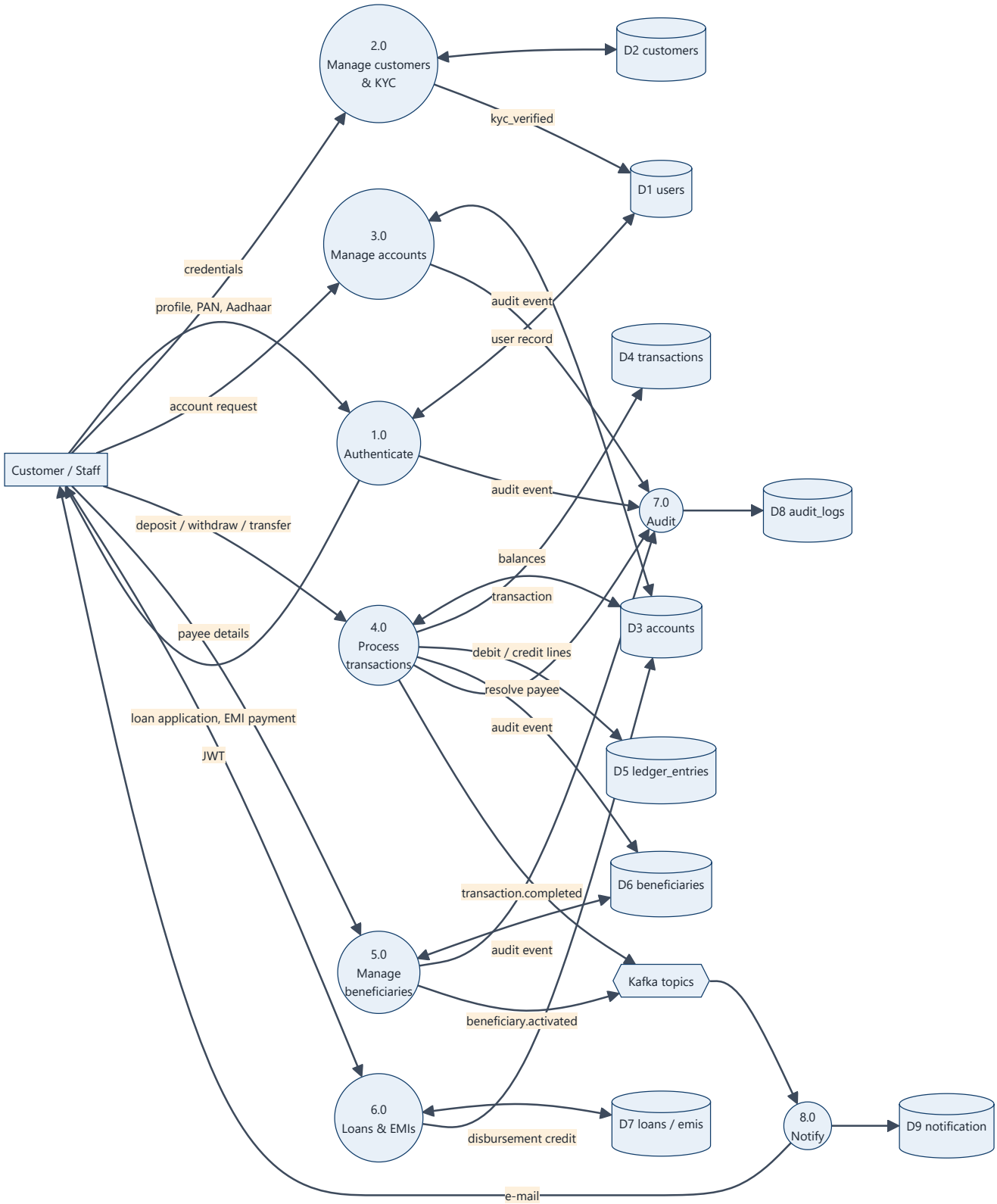
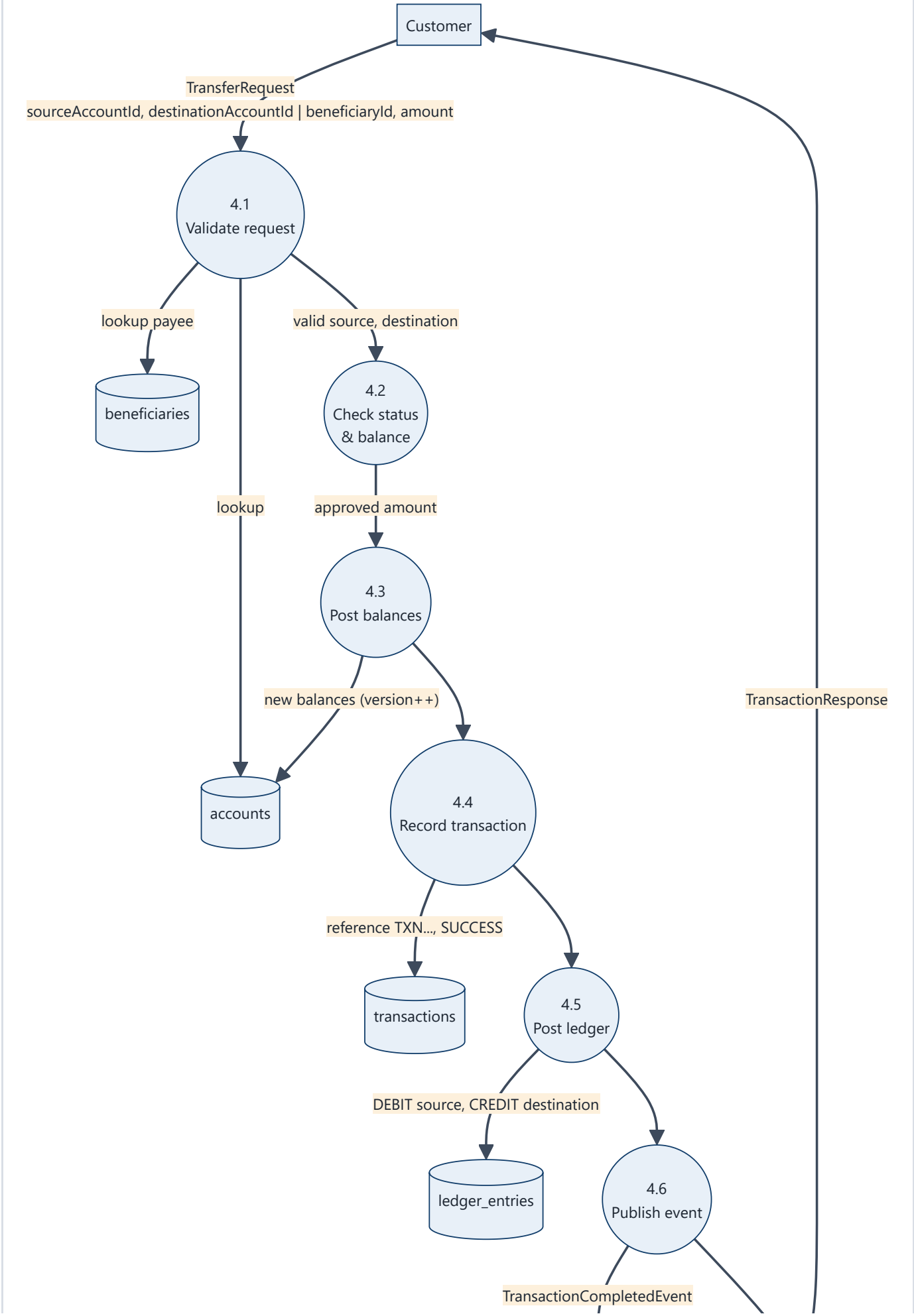


Figure 2 — DFD level 1

1.3 Level 2 — Process 4.0 “Process transactions” (transfer)


Customer
TransferRequest
sourceAccountId, destinationAccountId | beneficiaryId, amount
4.1
Validate request
lookup payee
valid source, destination
beneficiaries
4.2
Check status
& balance
lookup
approved amount
4.3
Post balances
new balances (version++)
TransactionResponse
accounts
4.4
Record transaction
reference TXN..., SUCCESS
transactions
4.5
Post ledger
DEBIT source, CREDIT destination
ledger_entries
4.6
Publish event
TransactionCompletedEvent

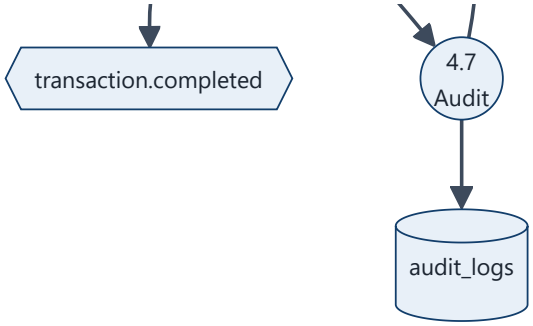


Figure 3 — DFD level 2 for a transfer

2. Sequence Diagrams — Authentication

2.1 Register

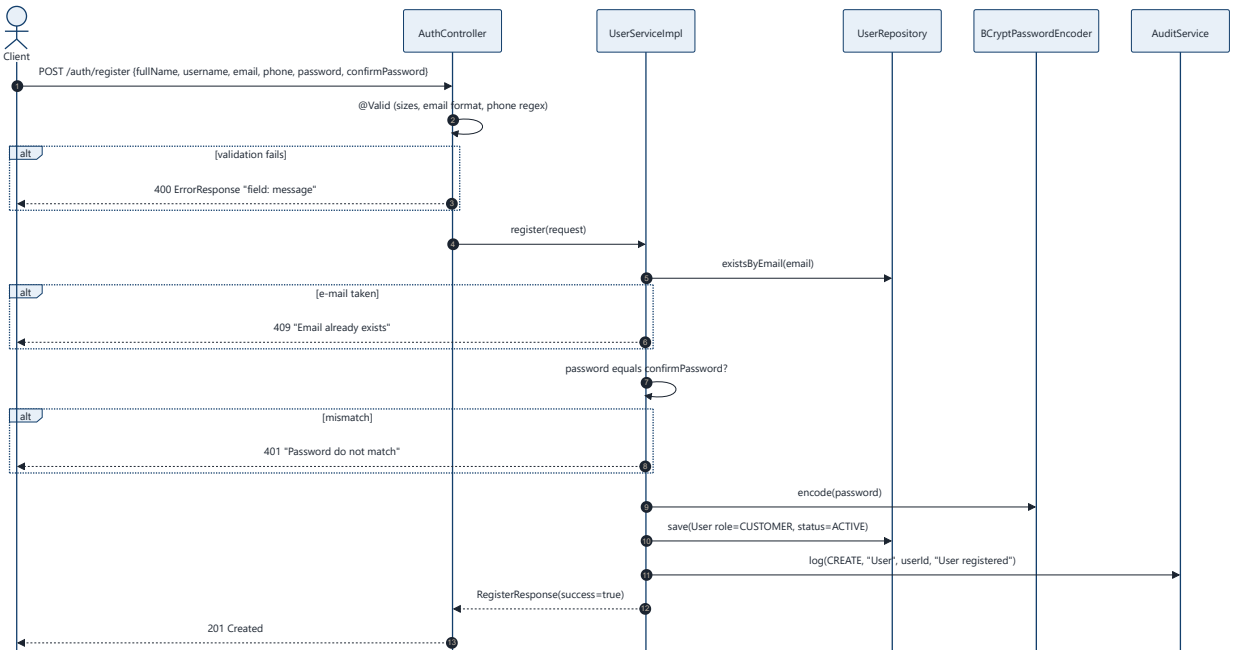


Figure 4 — Register

2.2 Login

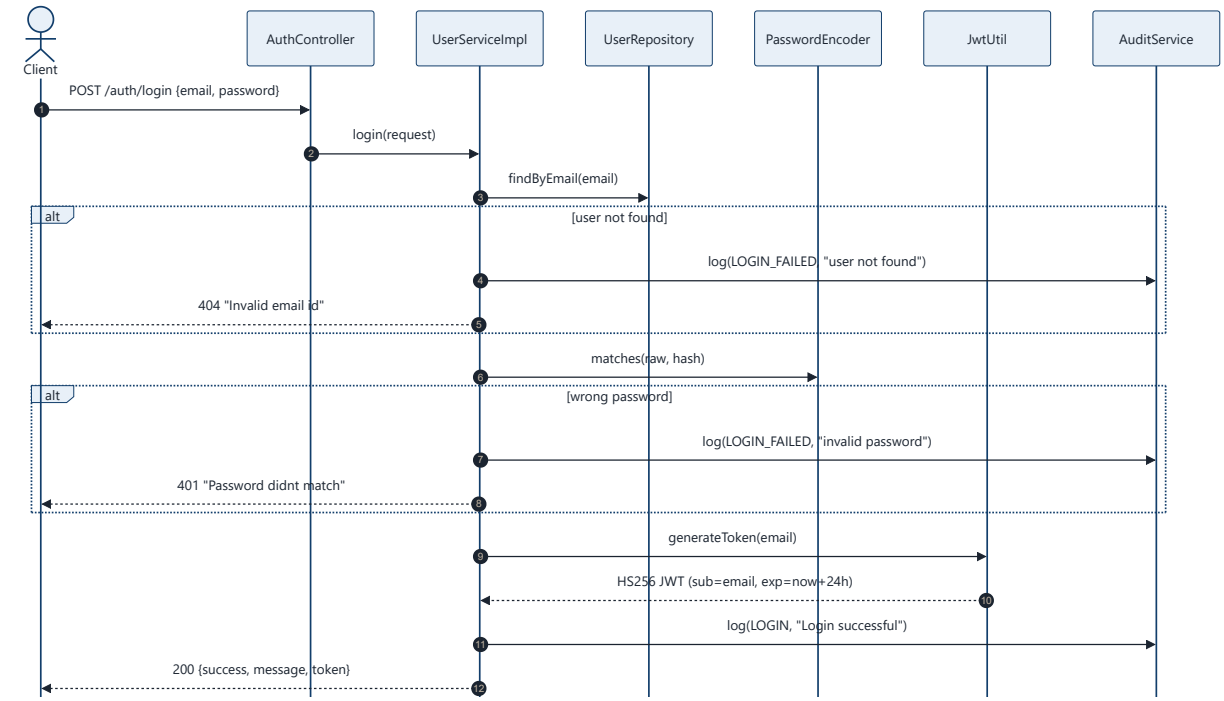


Figure 5 — Login

2.3 Authenticated request (JWT filter)

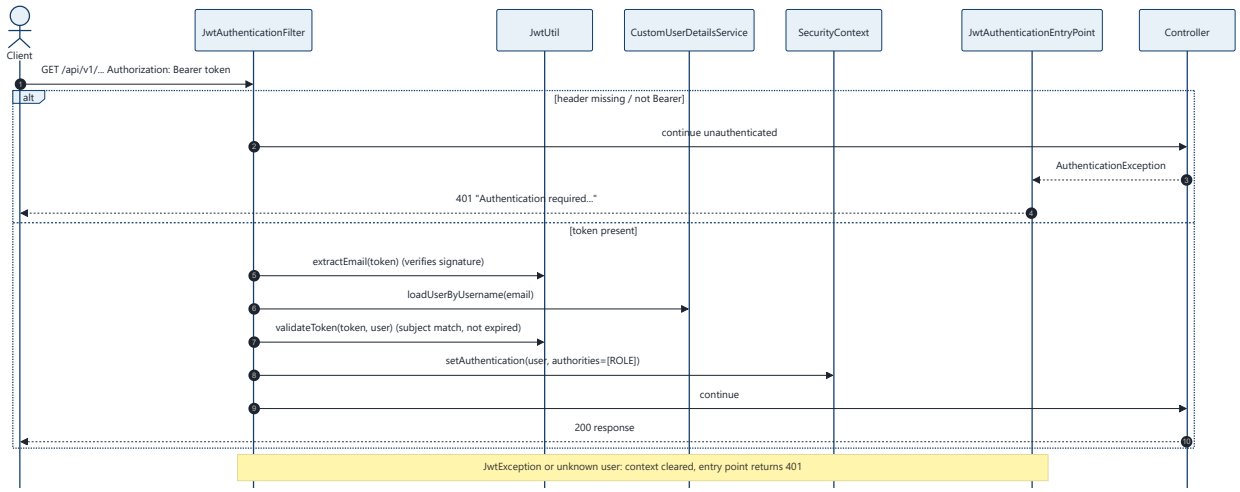


Figure 6 — Every protected request

3. Sequence Diagrams — Money Movement

3.1 Transfer (the core flow)

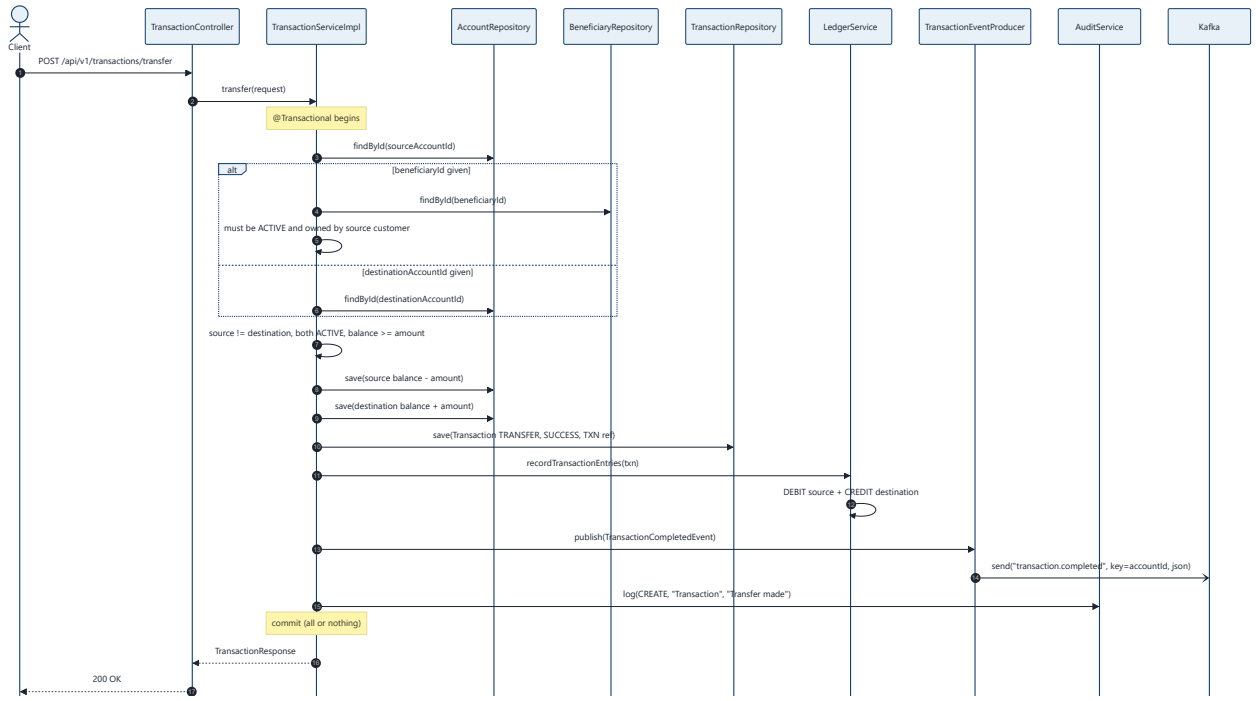


Figure 7 — Transfer. Any exception before commit rolls back balances, transaction, ledger and audit rows.

3.2 Deposit and withdrawal

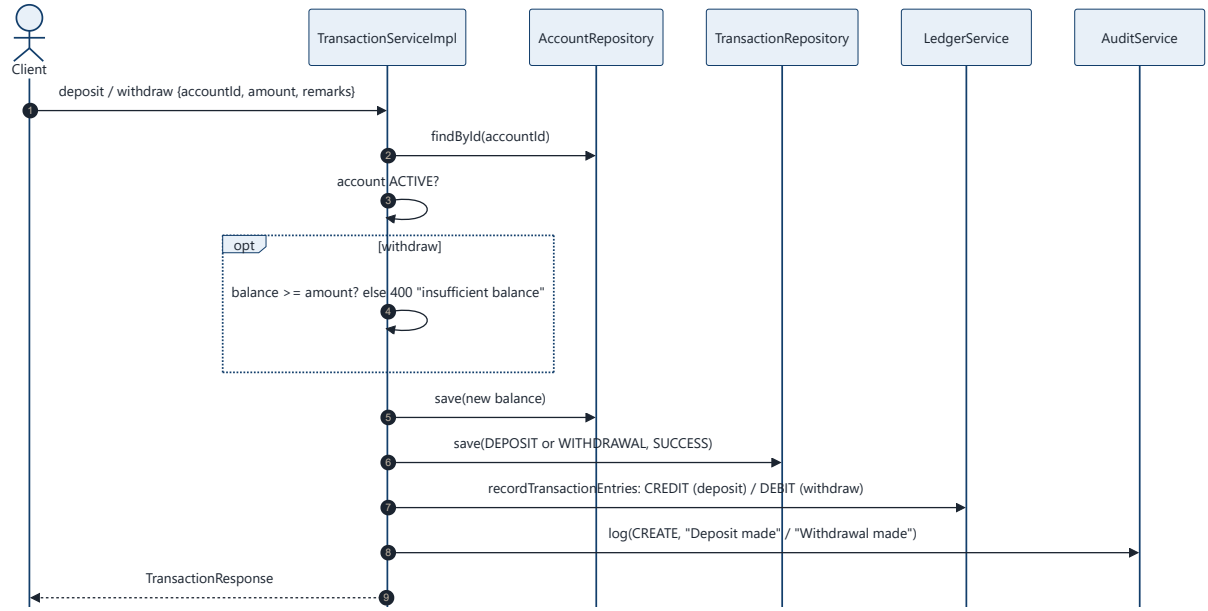


Figure 8 — Deposit / withdrawal (no Kafka event today)

4. Sequence Diagrams — Beneficiary, Loan and EMI

4.1 Add and activate beneficiary → notification

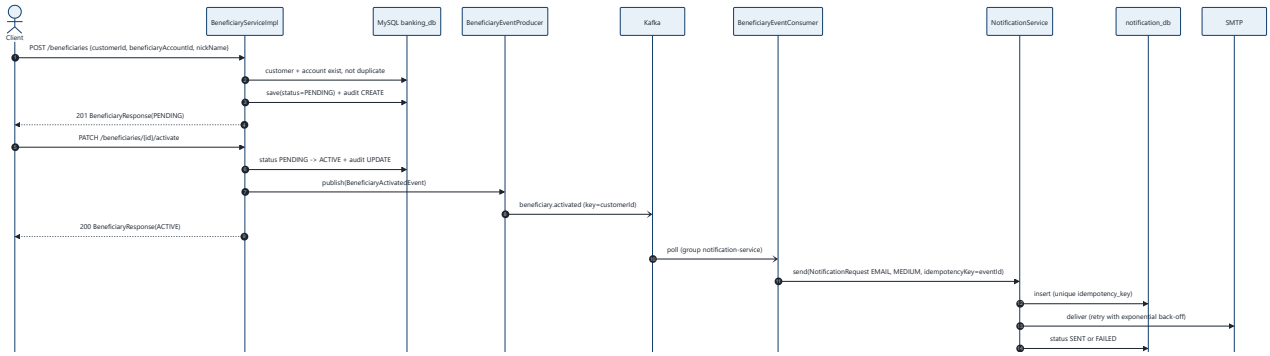


Figure 9 — Beneficiary lifecycle and asynchronous notification

4.2 Apply for and approve a loan

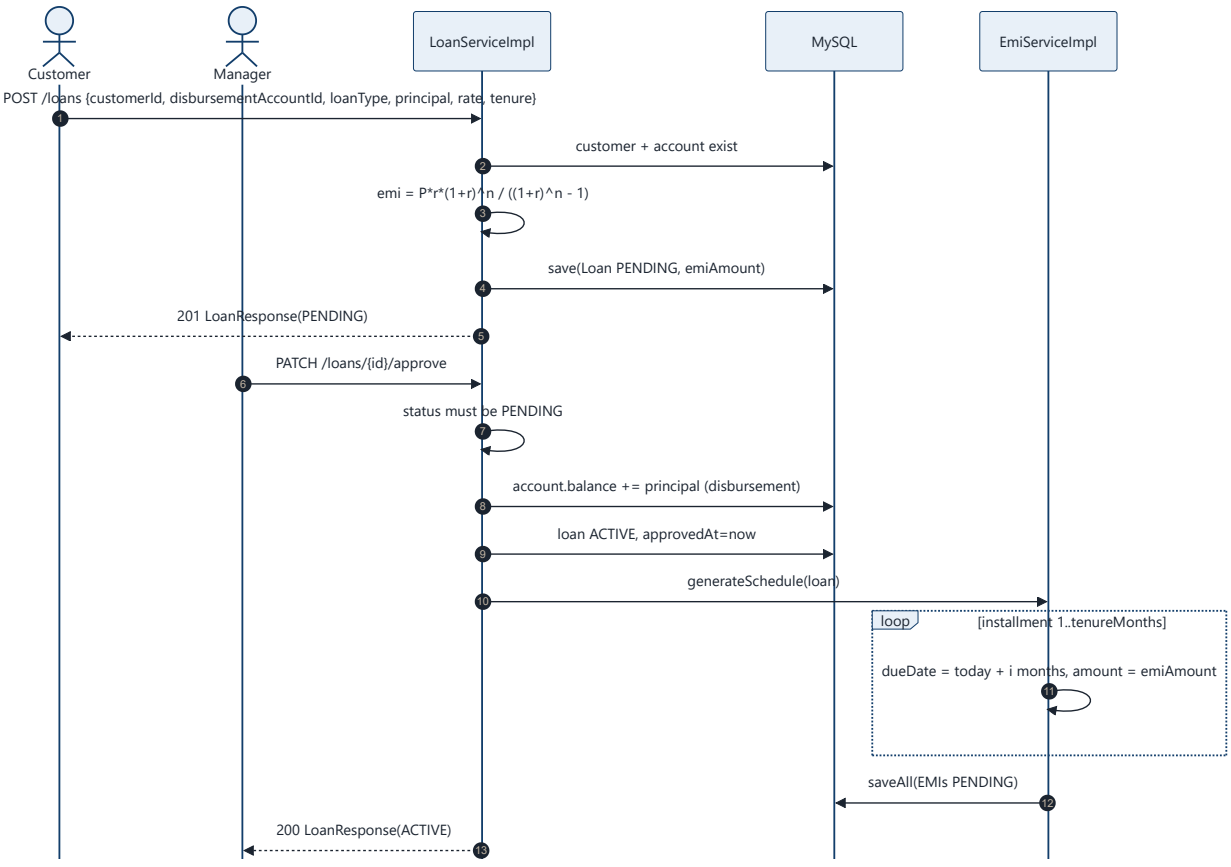


Figure 10 — Loan application and approval (reject: PENDING → REJECTED, nothing else)

4.3 Pay EMI

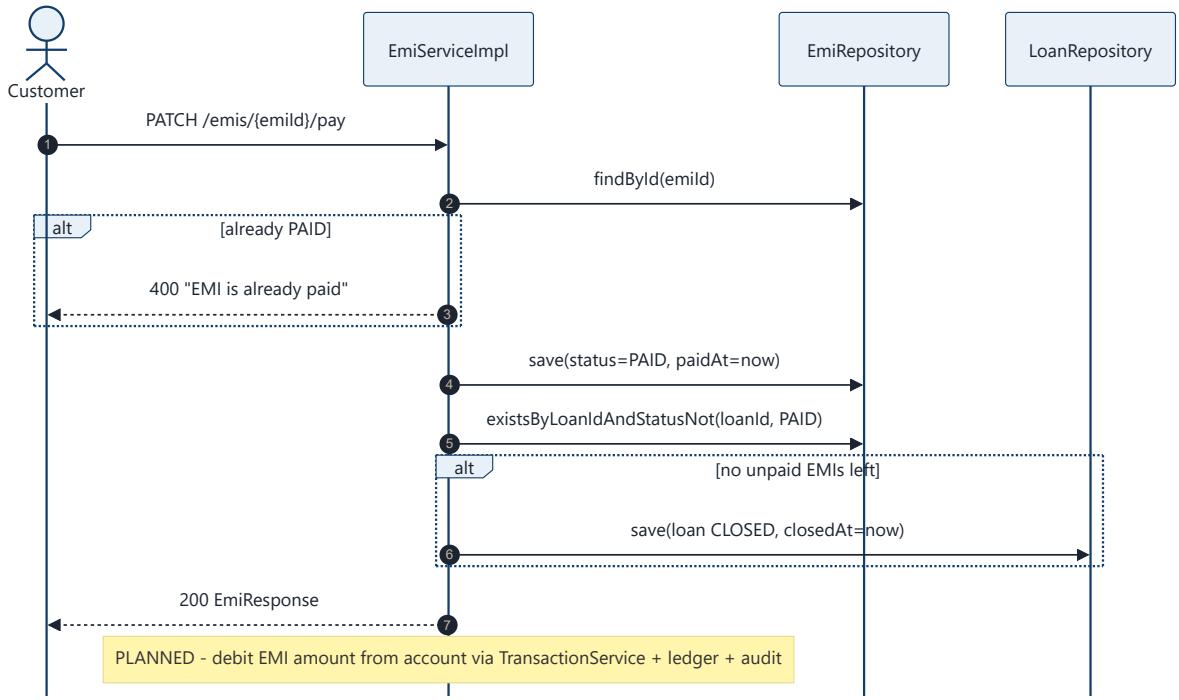
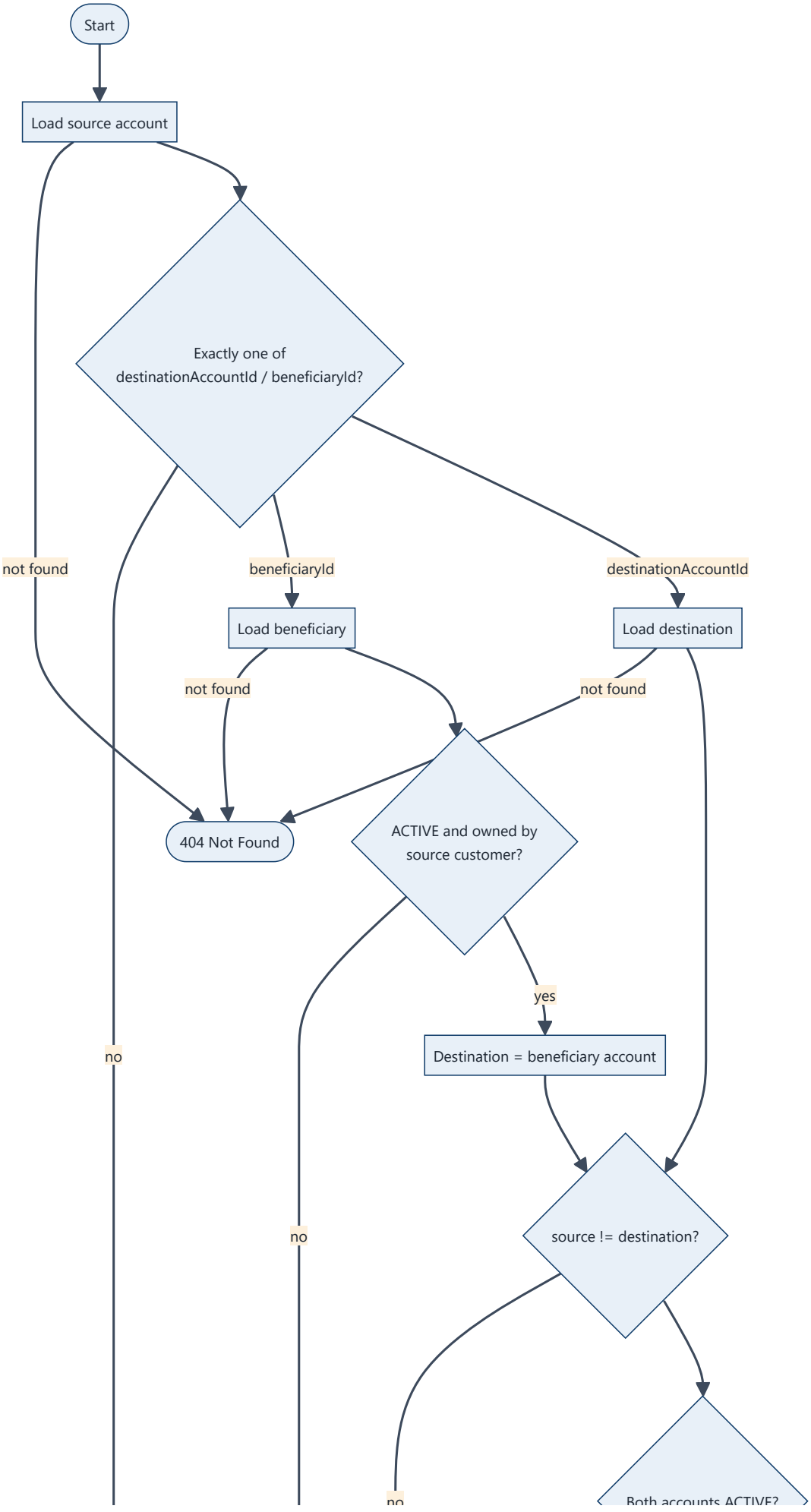


Figure 11 — EMI payment

5. Activity Diagrams

5.1 Transfer validation


Start
Load source account
Exactly one of destinationAccountId / beneficiaryId?
not found
beneficiaryId
destinationAccountId
Load beneficiary
Load destination
not found
not found
404 Not Found
ACTIVE and owned by source customer?
yes
no
Destination = beneficiary account
no
source != destination?
no
Both accounts ACTIVE?

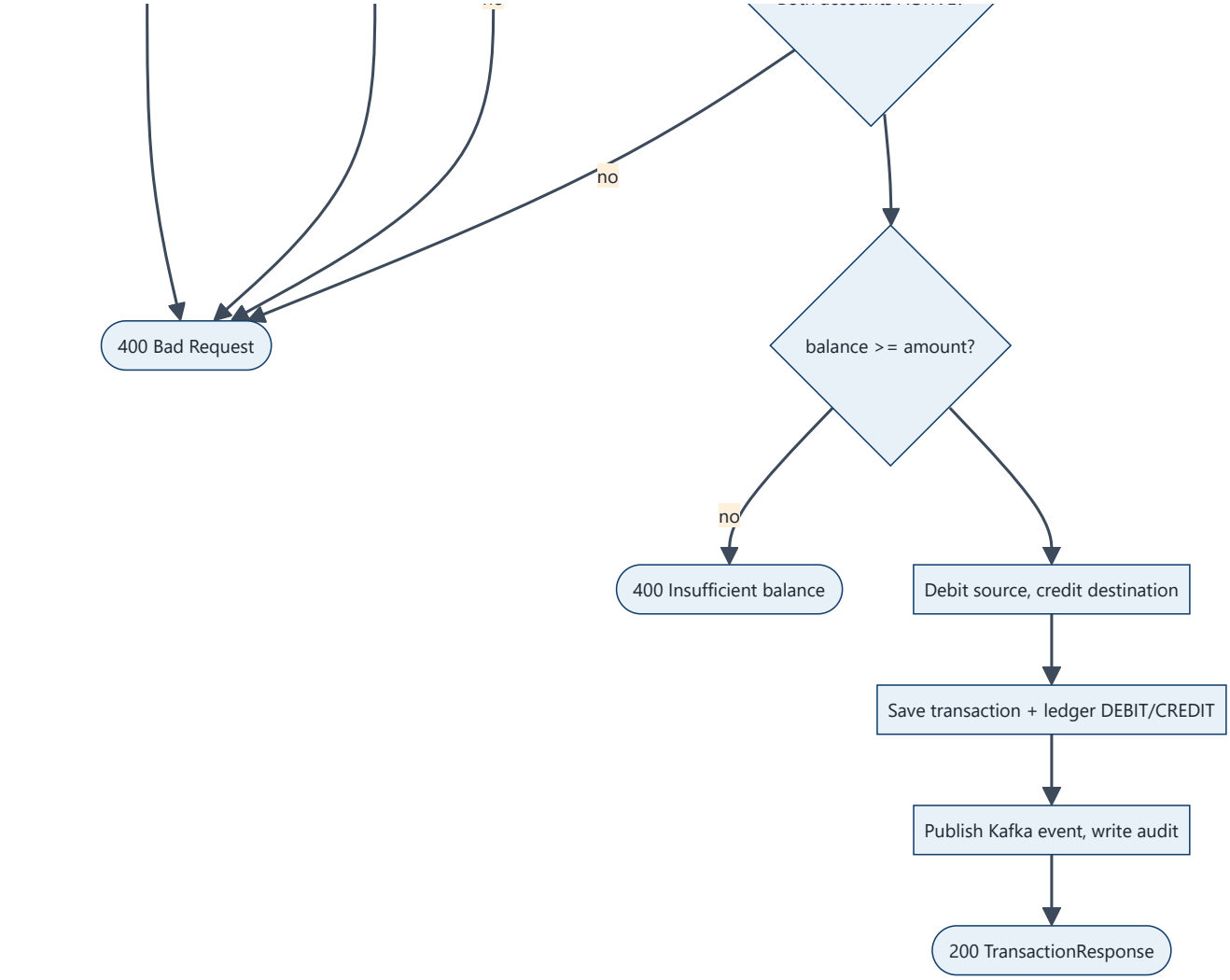


Figure 12 — Transfer activity

5.2 Customer onboarding (end to end)

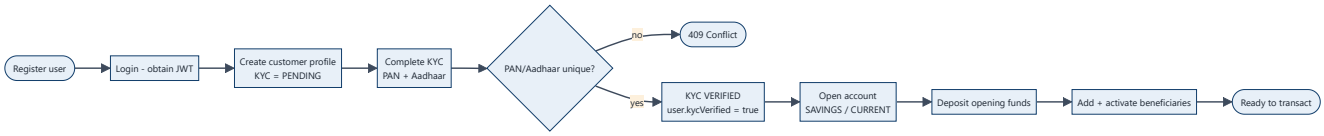


Figure 13 — Onboarding activity. Note: account opening does not yet require KYC = VERIFIED GAP

5.3 Overdue EMI job PLANNED

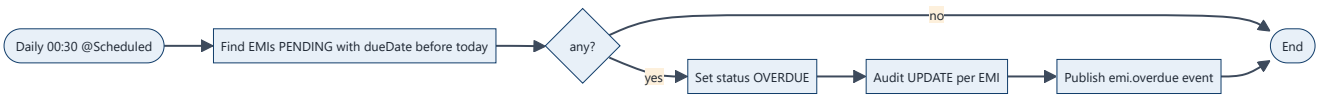


Figure 14 — Planned overdue detection

6. State Diagrams

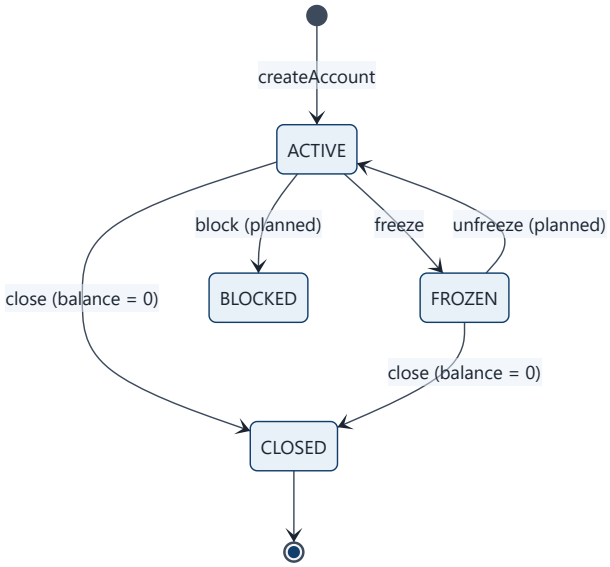


Figure 15 — Account. Only ACTIVE can transact; CLOSED is terminal.

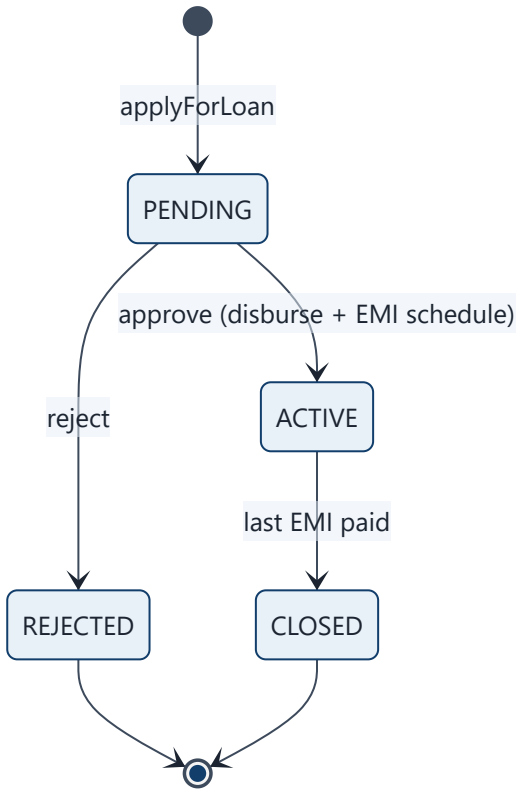


Figure 16 — Loan

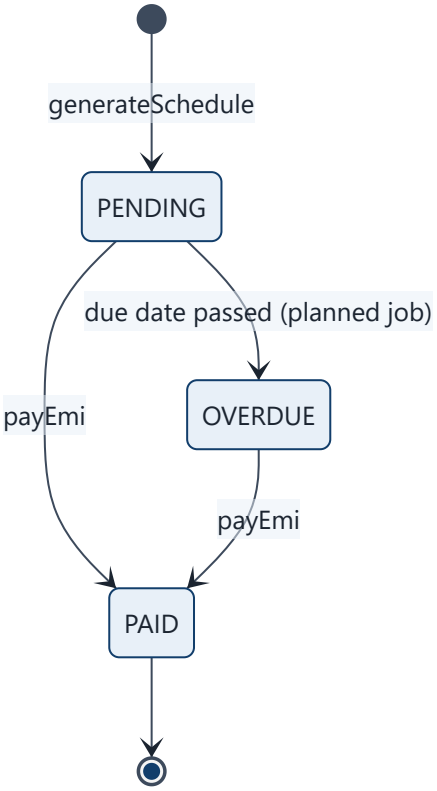


Figure 17 — EMI

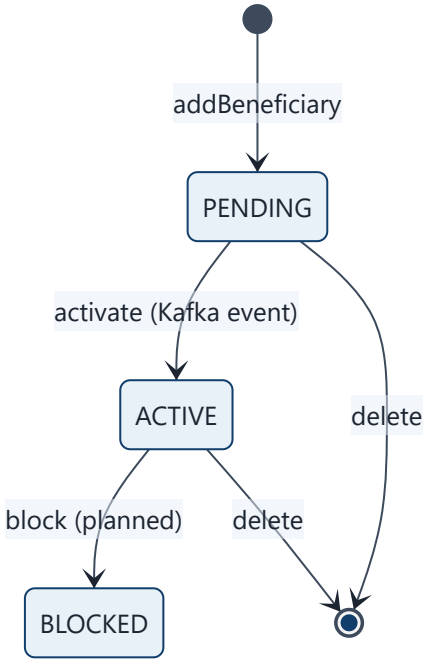


Figure 18 — Beneficiary

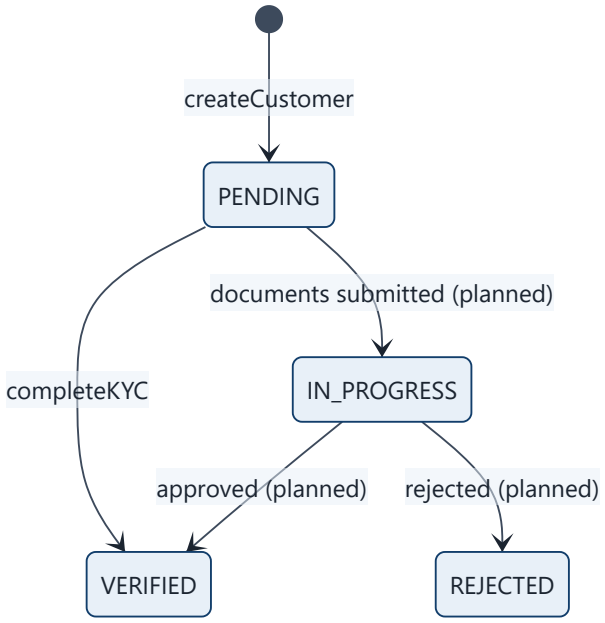


Figure 19 — Customer KYC

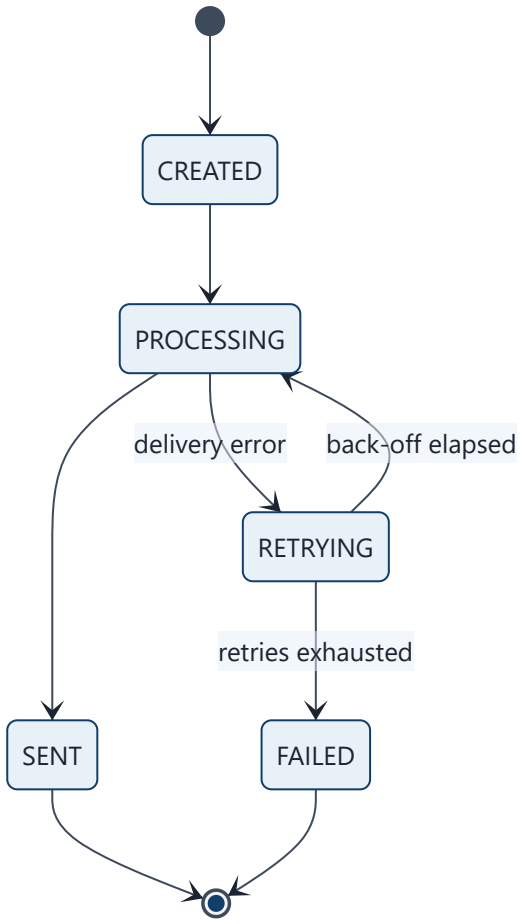


Figure 20 — Notification (notification-service)

6.1 Transaction status

Transactions are written directly as `SUCCESS` inside the same database transaction that moves the money; a failure rolls everything back, so no `FAILED` row is stored today. `PENDING` (asynchronous / inter-bank payments), `FAILED` (persisted failures) and `REVERSED` (compensating transaction) are reserved for future use.